

Projeto Integrador de Tecnologia da Informação III

Prof. Dr. Hudson Silva Borges

Módulo 2 - Teste de Software Aplicado

Unidade 2 - Projeto e desenvolvimento de testes

- Na unidade 1 revisamos conceitos importantes sobre testes.
- Nesta unidade, veremos como aplicar testes a partir de um estudo de caso.
- **Por que estudo de caso?** Pois estes oferecem uma visão mais próxima dos desafios práticos enfrentados em cenários reais.

Estudo de caso: **FACOFFEE**



- **FACOFFEE** é uma aplicação *web* para auxiliar na divisão dos custos do consumo de café e manutenção da máquina na copa da Faculdade de Computação da UFMS (FACOM/UFMS).
- Ela foi concebida e projetada como parte do processo de ensino de conceitos e técnicas de desenvolvimento, como Programação Funcional, *Domain Driven Design* (DDD) e Arquiteturas Baseadas em Serviços.

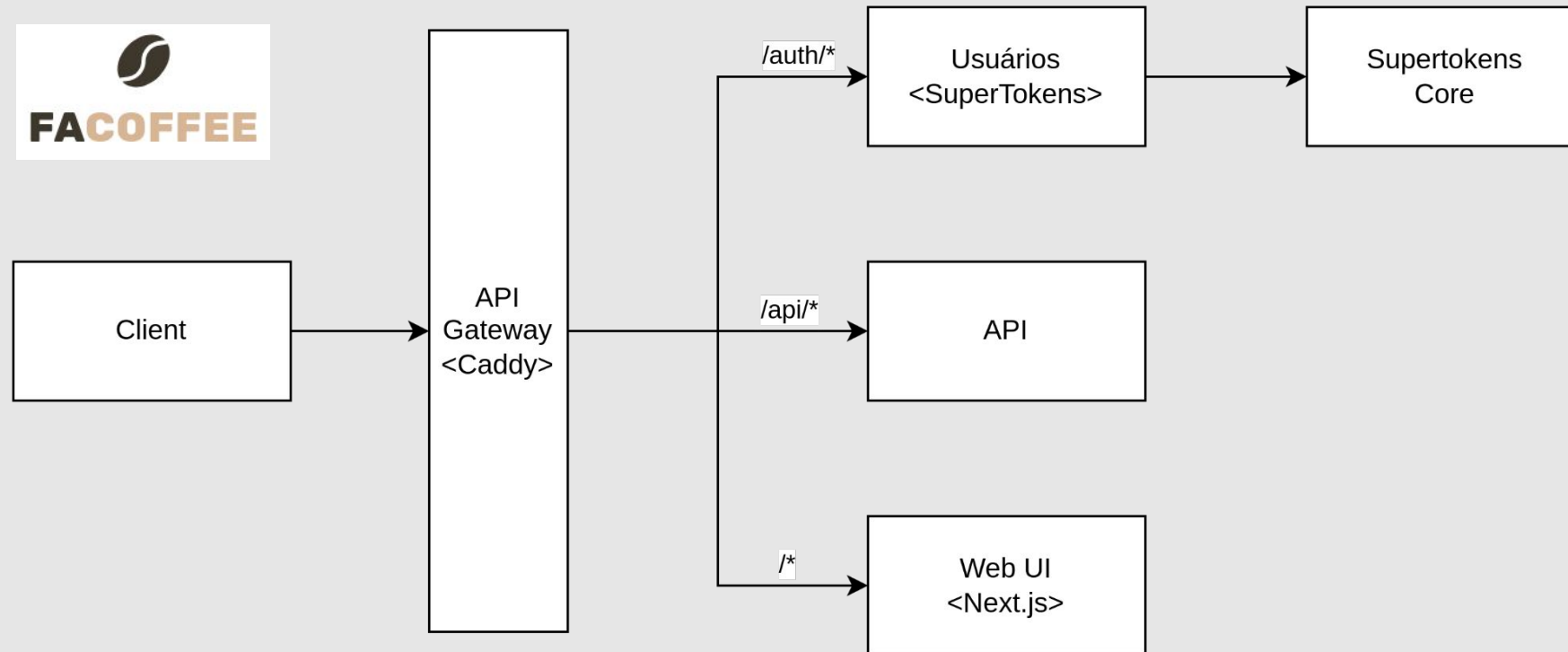
Estudo de caso: **FACOFFEE**

- Principais funcionalidades:

- Gestão de usuários (servidores);
- Autenticação e autorização;
- Gestão de planos de consumo;
- Gestão de assinaturas;
- Gestão de pagamentos (envio de comprovantes).

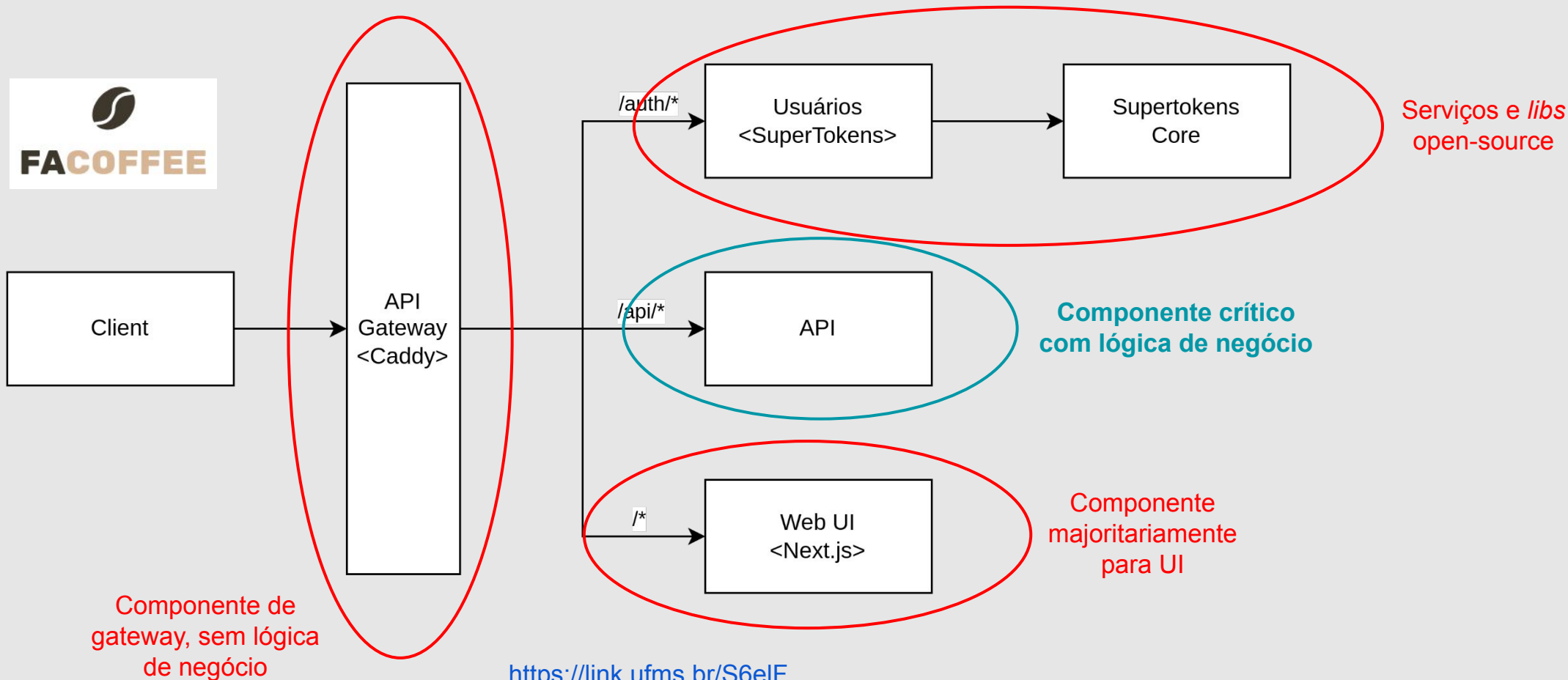


Estudo de caso: **FACOFFEE**



<https://link.ufms.br/S6eIF>
Fonte: Elaboração própria.

Por onde começar?



<https://link.ufms.br/S6eIF>
Fonte: Elaboração própria

- > .husky
- > .vscode
- > data
- ▼ src
 - > controllers
 - > entities
 - > middlewares
 - > services
 - > tasks
 - > utils
 - app.ts
 - global.d.ts
 - index.ts
 - .dockerignore
 - .editorconfig
 - .eslintignore

Por onde começar?

/controllers -> Tratamento de requisições http.

/entities -> Abstrações das entidades do domínio.

/middlewares -> Interceptações de requisições.

/services -> Atividades do negócio.

/tasks -> Scripts com procedimentos auxiliares.

/utils -> Funções utilitárias.

<https://github.com/hsborges/facoffee-api>

Seleção de ferramentas

JavaScript



Python



Java



JavaScript/Python/Independente



Criando testes

```
5 @Entity()
6 export class Plano {
7   @PrimaryGeneratedColumn('uuid')
8   public readonly id!: string;
9
10  @Column()
11  public readonly nome!: string;
12
13  @Column()
14  public readonly descricao!: string;
15
16  @Column('decimal', { precision: 10, scale: 2, transformer: new ColumnNumericTransformer() })
17  public readonly valor!: number;
18
19  @Column()
20  public readonly ativo!: boolean;
21
22  constructor(props: { nome: string; descricao: string; valor: number }) {
23    this.nome = props.nome;
24    this.descricao = props.descricao;
25    this.valor = props.valor;
26    this.ativo = true;
27  }
28
29  toJSON() {
30    return {
31      id: this.id,
32      nome: this.nome,
33      descricao: this.descricao,
34      valor: this.valor,
35      ativo: this.ativo,
36    };
37  }
38 }
```

```
1 import { faker } from '@faker-js/faker';
2
3 import { Plano } from './Plano';
4
5 describe('Testa entidade Plano', () => {
6   it('toJSON deve retornar um objeto com os dados', () => {
7     const data: ConstructorParameters<typeof Plano>[0] = {
8       nome: faker.string.sample(),
9       descricao: faker.string.sample(),
10      valor: faker.number.float(),
11    };
12    expect(new Plano(data).toJSON()).toEqual({ ...data, ativo: true });
13  });
14 });
```

Fonte: Elaboração própria

Executando e medindo progresso

```
> yarn jest
yarn run v1.22.22
$ /home/hudson/Github/facoffee/facoffee-api/node_modules/.bin/jest
PASS src/utils/transformer.spec.ts
PASS src/entities/Plano.spec.ts

Test Suites: 2 passed, 2 total
Tests:       5 passed, 5 total
Snapshots:   0 total
Time:        3.028 s, estimated 4 s
Ran all test suites.
Done in 3.35s.
```

Fonte: Elaboração própria.
Versão: 0.0.3

Expanda os testes

```
1 export abstract class HttpError extends Error {
2   public readonly code: number;
3   constructor(message: string, code: number) {
4     super(message);
5     this.code = code;
6   }
7 }
8
```

You, 15 months ago | 1 author (You)

```
9 export class NotFoundError extends HttpError {
10   constructor(message: string = 'Not found') {
11     super(message, 404);
12   }
13 }
14
```

You, 15 months ago | 1 author (You)

```
15 export class BadRequestError extends HttpError {
16   constructor(message: string = 'Validation error') {
17     super(message, 400);
18   }
19 }
20
```

You, 15 months ago | 1 author (You)

```
21 export class UnauthorizedError extends HttpError {
22   constructor(message: string = 'Unauthorized') {
23     super(message, 401);
24   }
25 }
26
```

You, 15 months ago | 1 author (You)

```
27 export class DuplicatedError extends HttpError {
28   constructor(message: string = 'Duplicated resource') {
29     super(message, 409);
30   }
31 }
```

```
12 it('deve retornar 404 para NotFoundError', () => {
13   expect(new NotFoundError()).toHaveProperty('code', 404);
14   expect(NotFoundError.name).toBe('NotFoundError');
15 });
16
17 it('deve retornar 409 para DuplicatedError', () => {
18   expect(new DuplicatedError()).toHaveProperty('code', 409);
19   expect(DuplicatedError.name).toBe('DuplicatedError');
20 });
21
22 it('deve retornar 401 para UnauthorizedError', () => {
23   expect(new UnauthorizedError()).toHaveProperty('code', 401);
24   expect(UnauthorizedError.name).toBe('UnauthorizedError');
25 });
26
```

Fonte: Elaboração própria

Executando e medindo progresso

```
> yarn jest
yarn run v1.22.22
$ /home/hudson/Github/facoffee/facoffee-api/node_modules/.bin/jest
PASS src/utils/transformer.spec.ts
PASS src/utils/error.spec.ts
PASS src/utils/wrapper.spec.ts
PASS src/entities/Plano.spec.ts (6.217 s)
PASS src/entities/Operacao.spec.ts (6.459 s)
PASS src/entities/Assinatura.spec.ts (6.711 s)
PASS src/entities/Credito.spec.ts (6.709 s)

Test Suites: 7 passed, 7 total
Tests:       26 passed, 26 total
Snapshots:   0 total
Time:        6.963 s
Ran all test suites.
Done in 7.27s.
```

Fonte: Elaboração própria.
Versão: 0.0.4

Executando e medindo progresso

All files

26.26% Statements 104/396 11.22% Branches 11/98 19.26% Functions 21/109 25.63% Lines 91/355

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File ▲		Statements ▾		Branches ▾		Functions ▾		Lines ▾	
src		0%	0/40	0%	0/10	0%	0/6	0%	0/38
src/controllers		0%	0/83	0%	0/22	0%	0/28	0%	0/77
src/entities		89.77%	79/88	100%	5/5	73.33%	11/15	91.66%	66/72
src/middlewares		0%	0/37	0%	0/6	0%	0/14	0%	0/34
src/services		0%	0/110	0%	0/45	0%	0/33	0%	0/97
src/tasks		0%	0/8	0%	0/2	0%	0/3	0%	0/7
src/utils		83.33%	25/30	75%	6/8	100%	10/10	83.33%	25/30

Fonte: Elaboração própria

Executando e medindo progresso

```
5   @Entity()
6   14x export class Plano {
7       @PrimaryGeneratedColumn('uuid')
8   14x   public readonly id!: string;
9
10      @Column()
11   14x   public readonly nome!: string;
12
13      @Column()
14   14x   public readonly descricao!: string;
15
16      @Column('decimal', { precision: 10, scale: 2, transformer: new ColumnNumericTransformer() })
17   14x   public readonly valor!: number;
18
19      @Column()
20   14x   public readonly ativo!: boolean;
21
22      constructor(props: { nome: string; descricao: string; valor: number }) {
23   28x         this.nome = props.nome;
24   28x         this.descricao = props.descricao;
25   28x         this.valor = props.valor;
26   28x         this.ativo = true;
27      }
28
29      toJSON() {
30   13x         return {
31             id: this.id,
32             nome: this.nome,
33             descricao: this.descricao,
34             valor: this.valor,
35             ativo: this.ativo,
36         };
37      }
38  }
39  }
```

Node/Jest

```
npx jest --coverage
```

Python/PyTes

```
pytest --cov=my_project --cov-report=html
```

Java/SpringBoot

```
./gradlew test jacocoTestReport
```

Fonte: Elaboração própria

Isolando unidades em testes unitários

- Muitas vezes será necessário criar *stubs*, *mocks* e *spys*.

```
const service = new OperacaoService({  
  repository: repository,  
  fileService: { save: jest.fn().mockResolvedValue(void 0) },  
});
```

Fonte: Elaboração própria

- Nesse caso, *OperacaoService* depende de *FileService*
- Podemos criar um *mock* para simular esse componente

Executando e medindo progresso

All files

62.87% Statements 249/396 **57.14%** Branches 56/98 **60.55%** Functions 66/109 **61.4%** Lines 218/355

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

Mais que dobramos a
cobertura somente com
testes unitários

File ▲		Statements ▾		Branches ▾		Functions ▾		Lines ▾	
src		0%	0/40	0%	0/10	0%	0/6	0%	0/38
src/controllers		0%	0/83	0%	0/22	0%	0/28	0%	0/77
src/entities		97.72%	86/88	100%	5/5	86.66%	13/15	97.22%	70/72
src/middlewares		62.16%	23/37	66.66%	4/6	71.42%	10/14	61.76%	21/34
src/services		100%	110/110	88.88%	40/45	100%	33/33	100%	97/97
src/tasks		0%	0/8	0%	0/2	0%	0/3	0%	0/7
src/utlis		100%	30/30	87.5%	7/8	100%	10/10	100%	30/30

Fonte: Elaboração própria
Versão: 0.0.6

Realizando testes de integração

- Testes de integração buscam verificar o comportamento dos componentes de forma integrada.
- Existem duas estratégias principais:
 - *Bottom-up* e *top-down*.
- Testes de API de forma *top-down* podem ser uma boa estratégia para iniciar com testes de integração.

- > .husky
- > .vscode
- > data
- ▼ src
 - > controllers
 - > entities
 - > middlewares
 - > services
 - > tasks
 - > utils
 - app.ts
 - global.d.ts
 - index.ts
 - .dockerignore
 - .editorconfig
 - .eslintignore

Estratégias?

/controllers -> Unitários/integração

/entities -> Unitários

/middlewares -> Unitários

/services -> Unitários

/tasks -> Unitários

/utils -> Unitários

Testes de integração

- Foco nos testes dos *endpoints/caminhos* do serviço.

```
TS PlanoController.spec.ts X
src > controllers > TS PlanoController.spec.ts > ...
13 describe('Testa PlanoController', () => {
21     describe('GET /', () => {
22         it('deve retornar vazio se não houver planos', async () => {
23             await supertest(app).get('/').expect(StatusCode.OK).expect([]);
24         });
25
26         it('deve retornar, por padrão, planos ativos', async () => {
27             let data = await repository.save(new Plano(sample));
28             await supertest(app).get('/').expect(StatusCode.OK).expect([data.toJSON()]);
29             data = await repository.save(repository.merge(data, { ativo: false }));
30             await supertest(app).get('/').expect(StatusCode.OK).expect([]);
31         });
32     });
}
```

Fonte: Elaboração própria

Executando e medindo progresso

All files

92.42% Statements 366/396 85.71% Branches 84/98 92.66% Functions 101/109 91.83% Lines 326/355

Cobrimos os principais
módulos do serviço

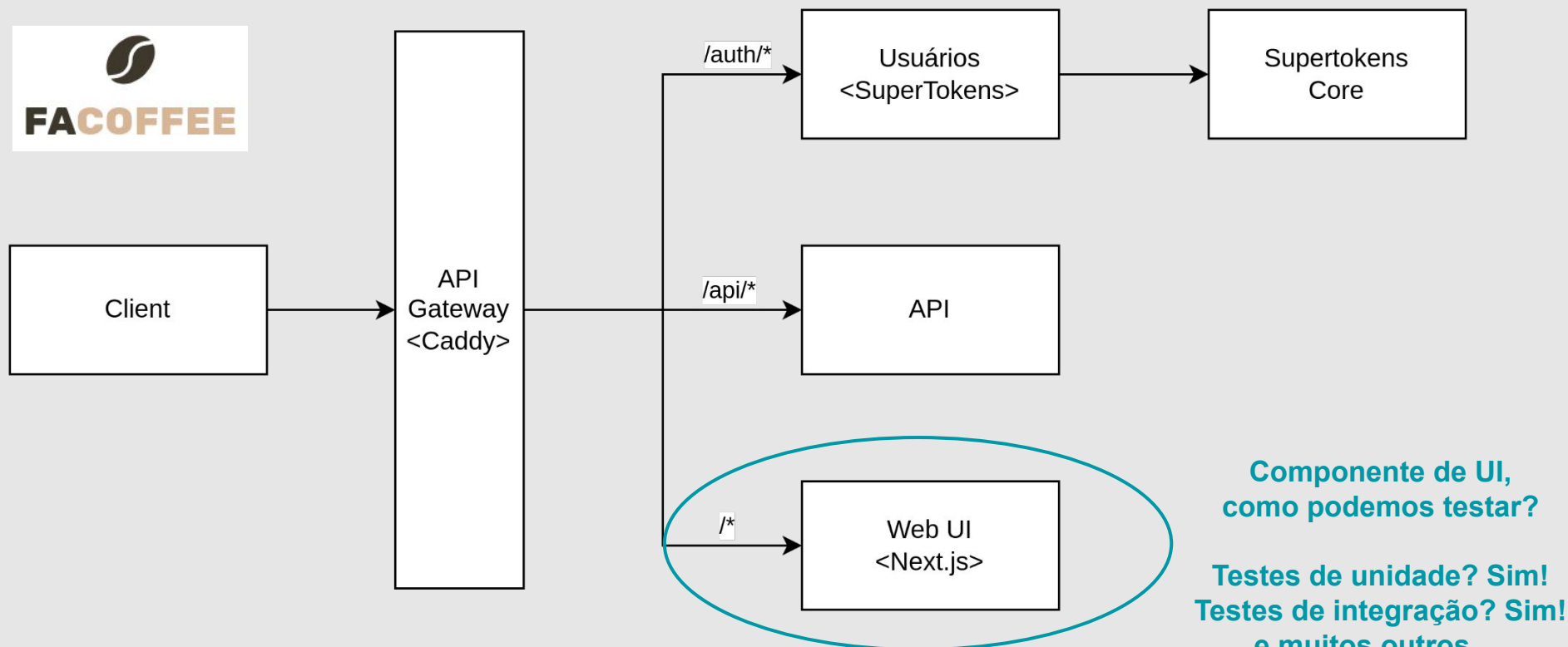
Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File ▲		Statements ⇅		Branches ⇅		Functions ⇅		Lines ⇅	
src		50%	20/40	20%	2/10	50%	3/6	47.36%	18/38
src/controllers		100%	83/83	95.45%	21/22	100%	28/28	100%	77/77
src/entities		100%	88/88	100%	5/5	100%	15/15	100%	72/72
src/middlewares		94.59%	35/37	83.33%	5/6	85.71%	12/14	94.11%	32/34
src/services		100%	110/110	97.77%	44/45	100%	33/33	100%	97/97
src/tasks		0%	0/8	0%	0/2	0%	0/3	0%	0/7
src/utls		100%	30/30	87.5%	7/8	100%	10/10	100%	30/30

Fonte: Elaboração própria
Versão: 0.0.8

Como avançar estrategicamente?



<https://link.ufms.br/S6eIF>
Fonte: Elaboração própria

Teste de sistema



Browser address bar: `npx playwright codegen demo.playwright.dev/todomvc`

Page content: `This is just a demo` | `real TodoMVC app.`

Page title: `todos`

Form input: `What needs to be done?`

Form input: `feed the dog`

Playwright Inspector window:

```
1 import { test, expect } from '@playwright/test';
2
3 test('test', async ({ page }) => {
4   await page.goto('https://demo.playwright.dev/todomvc/#/');
5   await page.getByPlaceholder('What needs to be done?').click();
6   await page.getByPlaceholder('What needs to be done?').fill('water the plants');
7   await page.getByPlaceholder('What needs to be done?').press('Enter');
8   await page.getByPlaceholder('What needs to be done?').fill('feed the dog');
9   await page.getByPlaceholder('What needs to be done?').press('Enter');
10  await page.locator('li').filter({ hasText: 'water the plants' }).getByLabel('Toggle Todo').check();
11  await page.getByRole('link', { name: 'Completed' }).click();
12  await expect(page.getByTestId('todo-title')).toContainText('water the plants');
13  await page.getByRole('link', { name: 'Active' }).click();
14  await expect(page.getByTestId('todo-title')).toContainText('feed the dog');
15 });
```

Locator Log

Conclusões

- *Frameworks* de testes ajudam na execução e análise.
- Dependências devem ter seu comportamento simulado.
- Nem todas as unidades “merecem” ser testadas.
- Componentes de alto impacto devem ser priorizados.
- Testes de integração ajudam na verificação de comportamentos quando estes são integrados e se comunicam entre si.

Atividade de *checkout*

- Você deverá criar um plano de testes para o sistema projetado em PI-I e desenvolvido em PI-II.
- Defina objetivos e descreva o escopo dos testes com base na proposta e arquitetura do sistema.
- Deixe claro no documento as estratégias a serem usadas e apresente casos de teste de exemplo para as estratégias escolhidas.

Atividade de *checkout*: algumas dicas

- Comece analisando os artefatos e a **arquitetura** projetada.
- Identifique os **principais componentes**, defina objetivos e proponha estratégias de testes.
- Não esqueça que você pode, e tipicamente irá, usar **diferentes técnicas** em seus testes!

Referências

DELAMARO, Márcio Eduardo; MALDONADO, José Carlos; JINO, Mario. **Introdução ao teste de software**. Rio de Janeiro: Gen Ltc, 2016. ISBN 9788595155732.

GONÇALVES, Priscila de Fátima et al. **Testes de software e gerência de configuração**. Porto Alegre: Sagah, 2019. ISBN 9788595029361.

SOMMERVILLE, Ian. **Engenharia de software**. 9. ed. São Paulo: Pearson, 2011. ISBN: 9788579361081.

VALENTE, Marco Tulio. **Engenharia de Software moderna**: princípios e práticas para desenvolvimento de software com produtividade. v.1, n.24, 2020. Disponível em: <https://link.ufms.br/bljNp>. Acesso em 20 dez 2024.

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).

